GradMap Product Requirements Document

- **Document Name:** GradMap
- **Document Version:** 1.0
- **Product Type:** Career and Master's Application Planning Workspace
- **Target Users:** Final-year undergraduate students (Final Year Students)
- **Primary Market Assumption:** Undergraduate students who need to prepare both postgraduate applications and post-graduation job searches, with priority support for Hong Kong and international application contexts
- **Document Status:** IDE-ready, for direct implementation by product, design, and engineering teams

---

1. Executive Summary

1.1 Product Overview

GradMap is a centralized career and master's application planning workspace that helps undergraduate graduates, within a single workspace:

1. Select one primary planning mode:
    - **Career Mode:** Manage graduate roles, companies, job requirements, and application deadlines.
    - **Master's Application Mode:** Manage universities, programmes, admission requirements, and application deadlines.
2. Compare the requirements of different programmes or roles.
3. Track application deadlines and progress.
4. Organize reusable documents, such as CVs, Personal Statements, recommendation letters, and transcripts.
5. Break each application into actionable tasks to prevent important steps from being missed.

Users select one mode during initial onboarding. The MVP permits only one mode per workspace to maintain a clear core experience; a future expansion may allow a single account to manage both modes.

1.2 Core Value Proposition

> GradMap enables students to consolidate application information scattered across university websites, job descriptions, document folders, and to-do lists into a single application plan that can be compared, tracked, and executed.

1.3 Target User Personas

Persona A: Master's Applicant — “The Planning-Oriented Student”

- **Background:** A final-year undergraduate student planning to apply to multiple universities and different master's programmes.
- **Pain Points:** Programme requirements, deadlines, document formats, and recommendation-letter requirements are fragmented; differences between individual programmes are easy to overlook.
- **Goal:** Quickly compare programmes and understand the completeness and outstanding tasks for each application.
- **Success Indicator:** Can view all programmes, deadlines, missing documents, and next actions in one dashboard.

Persona B: Graduate Job Seeker — “The Multi-Track Applicant”

- **Background:** Applying to multiple companies or graduate programmes.
- **Pain Points:** Different roles require different CVs, portfolios, assessments, and interview preparation; it is difficult to keep deadlines and application statuses up to date.
- **Goal:** Order job opportunities by deadline, reuse documents, and track each stage of the job-search process.
- **Success Indicator:** Every role has a clear status, deadline, and next task.

1.4 Monetization Mechanics

- **Model:** Subscription.
- **Trial:** The first month is free, conceptually similar to LinkedIn's introductory free period.
- **Billing Cycle:** Monthly or yearly.
- **Paywall:** Users have access to the full MVP feature set during the free first month; once the trial ends, a subscription is required to continue using protected workspace features.
- **Payment Flow:** Use Stripe Checkout only; GradMap must not handle card numbers directly.

- **Subscription Statuses:** `trialing`, `active`, `past_due`, `canceled`, `incomplete`, `incomplete_expired`.
- **MVP Pricing:** Driven by environment variables or Stripe Price IDs; prices must not be hardcoded in application logic. The frontend may display Stripe-configured monthly and yearly prices.
- **Downgrade Rule:** When a subscription expires, retain data but restrict creation and editing of workspace data; allow the user to view billing status and reactivate their subscription.
- **Refunds and Payment Disputes:** Do not build an in-product refund flow in the MVP; handle these through the Stripe Dashboard or a designated customer-support process.

1.5 MVP Success Metrics

- New users can complete registration, mode selection, and creation of their first application item within 10 minutes.
- Users can create at least 5 roles or programmes and see a list sorted by deadline.
- Users can create tasks for application items, mark them complete, and view remaining items.
- Users can upload and reuse at least one document.
- At least 80% of core actions can be completed using the keyboard and comply with WCAG 2.2 Level AA.
- Stripe webhooks reliably synchronize subscription status, avoiding reliance solely on the frontend payment-completion page.

---

2. Phased Scope Isolation

2.1 Phase 1 — Core MVP

A. Identity and Onboarding

- Clerk registration, sign-in, sign-out, and protected routes.
- First-login onboarding.
- Select one mode: `career` or `masters`.
- Create a personal workspace.
- Record timestamps of the user's agreement to the privacy notice, Terms of Service, and

marketing consent.

- Display the applicable PICS / privacy information disclaimer.

B. Workspace Dashboard

- Display the current mode name.
- Display the total number of application items, the number in progress, the number completed, and the number due soon.
- List application items sorted by deadline.
- Display each item's status, completion percentage, and next incomplete task.
- Display a primary “Add application item” CTA.
- Display empty-state guidance.

C. Career Mode

- Create, edit, and delete job applications.
- Fields include company, role title, location, job URL, deadline, application status, notes, and requirements.
- Supported statuses: `researching`, `ready_to_apply`, `applied`, `interviewing`, `offer`, `rejected`, `withdrawn`.
- Manage job requirements, such as education, skills, languages, portfolio, and assessments.
- Display a job detail page and associated tasks/documents.

D. Master's Application Mode

- Create, edit, and delete master's applications.
- Fields include university, programme name, location, programme URL, deadline, intake year, application status, notes, and requirements.
- Supported statuses: `researching`, `shortlisted`, `preparing`, `submitted`, `conditional_offer`, `offer`, `rejected`, `withdrawn`.
- Manage programme requirements, such as GPA, subject background, language scores, GRE/GMAT, recommendation letters, Personal Statement, and portfolio.
- Display a programme detail page and associated tasks/documents.

E. Requirements Comparison

- Select at least two application items within the same mode for comparison.

- Present requirement name, whether it is required, current completion status, and source notes in a table.
- On small screens, switch to a horizontally scrollable or stacked comparison-card layout.
- Do not automatically crawl third-party university or recruitment websites in the MVP; users manually enter information and source URLs.

F. Tasks and Deadline Tracking

- Create tasks for application items.
- Task fields: title, description, deadline, priority, status, and completion timestamp.
- Supported statuses: `todo`, `in_progress`, `done`.
- Supported priorities: `low`, `medium`, `high`.
- Display upcoming tasks on the dashboard.
- Users can create and complete tasks within the detail page.
- The MVP does not send email/push reminders; it must at minimum provide a prominent dashboard deadline indicator.

G. Reusable Documents

- Upload documents to ImageKit.
- Record document name, type, file size, MIME type, ImageKit file ID, URL, and upload timestamp.
- Supported document types: CV, Resume, Personal Statement, Cover Letter, Recommendation Letter, Transcript, Portfolio, Other.
- Link a document to one or more application items.
- Allow unlinking without deleting the original document.
- Allow users to delete their own documents and their corresponding ImageKit objects.
- MIME type, size, and file extension must be validated server-side.

H. Billing

- The pricing page displays monthly and yearly plans.
- Display the first-month-free trial message.
- Create the Stripe Checkout Session on the server.
- Return to the billing success page after subscription success, but treat the webhook as the source of truth for actual status.

- The billing page displays the current plan, status, and trial-end time.
- Provide entry to the Stripe Customer Portal, if enabled.

I. Non-Functional MVP Requirements

- Every protected data route must perform a Clerk identity check.
- Every data query must be scoped to the authenticated user ID / workspace ID.
- Any secret key may be used only server-side.
- The initial page and dashboard must support responsive desktop, tablet, and mobile layouts.
- Important mutations display loading, success, and error feedback.
- Use confirmation dialogs for destructive actions and support keyboard interaction.

2.2 Phase 2 — Post-Launch Expansion

The following features must not block the Phase 1 launch unless the scope is re-approved:

- Simultaneously enable Career Mode and Master's Application Mode for the same account.
- Usage credit analytics and an AI usage meter.
- Gemini AI assistance: extract requirements from user-provided programme/job descriptions, document gap analysis, and CV or Personal Statement suggestions.
- Multi-tenant team seats, teammate invitations, permissions, and shared workspaces.
- Career counselor / university advisor collaboration.
- Email, calendar, push notifications, and deadline reminders.
- CSV/JSON import/export.
- Automatically fetch and update programme/job requirements from public URLs (requires separate confirmation of robots, terms, and data-source compliance).
- Advanced filters, saved views, and a kanban board.
- Application analytics, such as conversion rate, average preparation time, and status funnel.
- Document version control, document diffing, and expiry reminders.
- Stripe coupons, promo codes, regional pricing, and invoicing workflows.
- Admin console, support tooling, and audit-log viewer.
- Multilingual content and locale-specific privacy flows.

2.3 Explicitly Out of Scope for the MVP

- Automatically submitting university or company applications on the user's behalf.
- Storing or processing third-party website login credentials.
- Automatically crawling restricted or login-required websites.
- Any guarantee of employment, admission, visa approval, or success rate.
- Interview video analysis, social matching, or job-recommendation algorithms.
- Team seats, administrator permissions, and cross-workspace sharing.
- AI-generated formal documents that can be submitted without user review.

---

3. Architecture & UI Layout

3.1 Page Routes

Route	Access	Purpose
`/`	Public	Marketing landing page, presenting the core value proposition and primary CTA
`/pricing`	Public	Monthly/yearly plans, first-month-free explanation, and FAQ
`/privacy`	Public	Privacy Policy, PICS, and data-processing explanation
`/terms`	Public	Terms of Service
`/sign-in`	Public	Clerk sign-in
`/sign-up`	Public	Clerk registration
`/onboarding`	Authenticated	Mode selection, consent notices, and workspace creation
`/app`	Authenticated + Subscription gate	Dashboard
`/app/applications`	Authenticated + Subscription gate	Application-item list

`/app/applications/new`	Authenticated + Subscription gate	Create application item
`/app/applications/[id]`	Authenticated + Subscription gate	Application item details, requirements, tasks, and documents
`/app/compare`	Authenticated + Subscription gate	Compare two or more application items
`/app/documents`	Authenticated + Subscription gate	Reusable document library
`/app/settings`	Authenticated	Profile, mode, and privacy settings
`/app/billing`	Authenticated	Subscription, trial, and payment status
`/checkout/success`	Authenticated	Checkout return page; displays webhook synchronization pending state
`/checkout/cancel`	Authenticated	Return page after the user cancels Checkout
`/api/stripe/checkout`	Server route	Create Checkout Session
`/api/stripe/portal`	Server route	Create Customer Portal Session
`/api/stripe/webhook`	Public server endpoint	Receive and verify Stripe webhooks
`/api/upload/imagekit`	Authenticated server route	Generate ImageKit upload authentication parameters

3.2 User Journey

First Use

1. The user selects “Get started” from the landing page.
2. The user completes registration or sign-in with Clerk.
3. The system checks whether the user has a workspace.
4. The user reads and accepts the Terms, Privacy/PICS, and required data-processing notices.

5. The user selects `Career Mode` or `Master's Application Mode`.
6. The system creates a workspace and redirects to `/app`.
7. The dashboard displays a mode-specific empty state and an “Add your first application” CTA.
8. The user creates their first job or programme.
9. The user adds requirements, tasks, and documents.
10. The dashboard reflects deadlines and completion percentage.

Ongoing Use

1. The user enters the dashboard after signing in.
2. The user reviews upcoming applications and tasks.
3. The user opens an application detail page.
4. The user updates a requirement, task, or status.
5. The user selects a reusable document from the document library and links it to an application.
6. The user uses the compare page to review different options.
7. If the trial is about to end, the system displays a non-blocking billing notice.
8. The user selects monthly or yearly billing on the pricing/billing page and proceeds to Stripe Checkout.
9. The Stripe webhook updates the local subscription state.

3.3 Key UI Components

Landing Page Hero

- Clear headline: Centrally manage your master's applications or graduate job-search plan.
- Supporting copy explains the two modes and core workflow.
- Primary CTA: `Start your free month`.
- Secondary CTA: `See how it works`.
- Optional product-preview screenshot/mockup.
- Do not make misleading claims about success rates or employment guarantees.

Mode Selection

- Two selectable cards: Career Mode and Master's Application Mode.
- Each card includes an icon, purpose, principal data managed, and examples.
- The selected state must provide textual and visual feedback; it cannot rely on color alone.
- Provide “You can change this later in settings” or clearly explain the MVP mode limitation.

Workspace Dashboard

- Header: workspace name, mode badge, and user menu.
- Summary cards: Total, In progress, Due soon, Completed.
- Primary CTA: New application.
- Deadline list/table.
- Upcoming tasks panel.
- Recent documents panel.
- Empty state: guide the user through three steps to create their first plan.

Application Card / Table

- Name, company/university, deadline, status, completion percentage, and next task.
- Display status with a badge and provide a text label.
- Display deadlines using relative time plus the absolute date, for example, “In 5 days · 2026-11-12”.
- Provide view, edit, and delete actions.

Application Detail

- Overview header: title, organization, deadline, and status.
- Tabs or sections: Overview, Requirements, Tasks, Documents, Notes.
- A progress indicator displays requirement/task completion percentage.
- A sticky or prominent next action.
- On mobile, use stacked sections.

Requirements Comparison

- Multi-select application picker.
- Desktop table, with mobile horizontal-scroll/card fallback.

- Columns: Requirement, Required, Application A, Application B, Notes/source.
- Display missing data and inconsistencies.

Pricing Table

- Monthly and Yearly plan cards.
- Display the first month free, subsequent recurring price, included features, and cancellation method.
- Avoid hiding material terms; clearly state when charges begin after the trial ends.
- CTA initiates a server-side Stripe Checkout session.
- Display currency, tax, or billing caveats according to Stripe configuration and the applicable region.

Feedback Components

- Inline validation.
- Toasts are supplementary only and must not be the sole error message.
- Loading skeletons.
- Empty states.
- Error boundary / retry.
- Destructive-action confirmation dialog.

---

4. Technical Stack Constraints

4.1 Required Stack

- **Framework:** Next.js with App Router.
- **Language:** TypeScript with strict mode enabled.
- **Styling:** Tailwind CSS.
- **ORM:** Drizzle ORM.
- **Database:** Neon PostgreSQL.
- **Authentication:** Clerk Auth.
- **Payments:** Stripe Checkout and Stripe Webhooks.
- **File storage / image delivery:** ImageKit.

- **AI integration:** Gemini API, the default for Phase 2; Phase 1 retains only a server-side integration boundary and must not treat AI as a launch blocker.

4.2 Architectural Rules

- Use Server Components by default; use Client Components only when client state, browser APIs, or event handlers are needed.
- Route all database mutations through a server action or authenticated route handler.
- Verify every Clerk identity server-side.
- Never use Neon credentials, Stripe secrets, ImageKit private keys, or Gemini API keys directly from the client.
- Validate all inputs with a schema (Zod recommended) before they enter a Drizzle query.
- Database queries must use parameterized queries / the Drizzle query builder; SQL string concatenation is prohibited.
- Place the subscription gate in the server-side authorization layer; do not merely hide frontend buttons.
- Store timestamps in UTC; format them for display according to the user's locale/time zone.
- Manage schema changes through migrations; production runtime must not modify the schema automatically.

4.3 Recommended Project Structure

Plain Text

```
app/
  (marketing)/
  (auth)/
  (dashboard)/app/
  api/
    stripe/
      checkout/route.ts
      portal/route.ts
      webhook/route.ts
    upload/imagekit/route.ts
components/
  ui/
  marketing/
  dashboard/
  applications/
  billing/
```

```
  documents/
db/
  schema.ts
  index.ts
  migrations/
lib/
  auth.ts
  billing.ts
  imagekit.ts
  validation.ts
  permissions.ts
  privacy.ts
```

---

5. Database & Authentication Schema

5.1 General Database Rules

- All primary keys use PostgreSQL `uuid`, defaulting to `gen_random_uuid()`.
- All user-owned entities must be isolated through `workspace_id` or an owner relationship.
- All foreign keys use `ON DELETE CASCADE` by default, unless otherwise stated below.
- All timestamps use `timestamptz`.
- Use `text` to store variable-length text; statuses may use a PostgreSQL enum or validated text. To support evolution, the MVP should use text plus application validation or enums that can be safely migrated.
- Important mutations should include `updated_at` and an optimistic-concurrency strategy.

5.2 Tables

`users`

Clerk is the identity source; this table stores only the profile mirror required by the application.

Column	Type	Constraints
`id`	`uuid`	PK, default `gen_random_uuid()`
`clerk_user_id`	`text`	UNIQUE, NOT NULL

email	text	NOT NULL
display_name	text	NULLABLE
timezone	text	NOT NULL, default UTC
created_at	timestamptz	NOT NULL, default now()
updated_at	timestamptz	NOT NULL, default now()

workspaces

Column	Type	Constraints
id	uuid	PK, default gen_random_uuid()
owner_user_id	uuid	FK users.id , NOT NULL, ON DELETE CASCADE
name	text	NOT NULL
mode	text	NOT NULL, CHECK IN (career , masters)
created_at	timestamptz	NOT NULL, default now()
updated_at	timestamptz	NOT NULL, default now()

Create a unique constraint on owner_user_id in the MVP to ensure each user has only one workspace; remove this restriction in Phase 2 to support multiple workspaces/team seats.

applications

Column	Type	Constraints
id	uuid	PK, default gen_random_uuid()
workspace_id	uuid	FK workspaces.id , NOT NULL, ON DELETE CASCADE
type	text	NOT NULL, CHECK IN (career , masters)
organization_name	text	NOT NULL

title	text	NOT NULL
location	text	NULLABLE
source_url	text	NULLABLE
deadline_at	timestamptz	NULLABLE
status	text	NOT NULL
intake_year	integer	NULLABLE; primarily used for masters
notes	text	NULLABLE
created_at	timestamptz	NOT NULL, default now()
updated_at	timestamptz	NOT NULL, default now()

Application type must match workspace mode ; enforce this through server-side validation and a database trigger/check strategy.

requirements

Column	Type	Constraints
id	uuid	PK, default gen_random_uuid()
application_id	uuid	FK applications.id , NOT NULL, ON DELETE CASCADE
name	text	NOT NULL
description	text	NULLABLE
is_required	boolean	NOT NULL, default true
status	text	NOT NULL, CHECK IN (unknown , not_started , in_progress , met , not_met , not_applicable)
source_note	text	NULLABLE
created_at	timestamptz	NOT NULL, default now()

updated_at	timestamptz	NOT NULL, default now()

tasks

Column	Type	Constraints
id	uuid	PK, default gen_random_uuid()
application_id	uuid	FK applications.id, NOT NULL, ON DELETE CASCADE
title	text	NOT NULL
description	text	NULLABLE
due_at	timestamptz	NULLABLE
priority	text	NOT NULL, CHECK IN (low , medium , high)
status	text	NOT NULL, CHECK IN (todo , in_progress , done)
completed_at	timestamptz	NULLABLE
created_at	timestamptz	NOT NULL, default now()
updated_at	timestamptz	NOT NULL, default now()

documents

Column	Type	Constraints
id	uuid	PK, default gen_random_uuid()
workspace_id	uuid	FK workspaces.id, NOT NULL, ON DELETE CASCADE
uploaded_by_user_id	uuid	FK users.id, NOT NULL, ON DELETE CASCADE
name	text	NOT NULL
document_type	text	NOT NULL

mime_type	text	NOT NULL
file_size_bytes	bigint	NOT NULL
imagekit_file_id	text	UNIQUE, NOT NULL
imagekit_url	text	NOT NULL
created_at	timestamptz	NOT NULL, default now()
updated_at	timestamptz	NOT NULL, default now()

application_documents

Column	Type	Constraints
application_id	uuid	Composite PK; FK applications.id, ON DELETE CASCADE
document_id	uuid	Composite PK; FK documents.id, ON DELETE CASCADE
created_at	timestamptz	NOT NULL, default now()

subscriptions

Column	Type	Constraints
id	uuid	PK, default gen_random_uuid()
workspace_id	uuid	FK workspaces.id, UNIQUE, NOT NULL, ON DELETE CASCADE
stripe_customer_id	text	UNIQUE, NOT NULL
stripe_subscription_id	text	UNIQUE, NULLABLE
stripe_price_id	text	NULLABLE
status	text	NOT NULL
trial_start_at	timestamptz	NULLABLE

trial_end_at	timestamptz	NULLABLE
current_period_start_at	timestamptz	NULLABLE
current_period_end_at	timestamptz	NULLABLE
cancel_at_period_end	boolean	NOT NULL, default false
created_at	timestamptz	NOT NULL, default now()
updated_at	timestamptz	NOT NULL, default now()

consents

Column	Type	Constraints
id	uuid	PK, default gen_random_uuid()
user_id	uuid	FK users.id , NOT NULL, ON DELETE CASCADE
consent_type	text	NOT NULL, for example terms , privacy , pics , marketing
policy_version	text	NOT NULL
granted	boolean	NOT NULL
consented_at	timestamptz	NOT NULL
withdrawn_at	timestamptz	NULLABLE
ip_hash	text	NULLABLE; unassessed plaintext IP addresses must not be stored
user_agent_hash	text	NULLABLE

stripe_events

Used for webhook idempotency.

Column	Type	Constraints
id	uuid	PK, default gen_random_uuid()

stripe_event_id	text	UNIQUE, NOT NULL
event_type	text	NOT NULL
processed_at	timestamptz	NULLABLE
payload_hash	text	NOT NULL
created_at	timestamptz	NOT NULL, default now()

5.3 Indexes

- workspaces.owner_user_id unique index.
- applications.workspace_id, deadline_at composite index.
- applications.workspace_id, status composite index.
- requirements.application_id index.
- tasks.application_id, status, due_at composite index.
- documents.workspace_id, created_at composite index.
- subscriptions.stripe_customer_id unique index.
- stripe_events.stripe_event_id unique index.

5.4 Authentication and Authorization

- Clerk is the authentication provider; clerk_user_id is the external identity key.
- Obtain Clerk session claims server-side for every request.
- Resolve the local users.id from clerk_user_id .
- For every query, first obtain the user's workspace, then execute within that workspace scope.
- The MVP has only a workspace owner; add membership/role tables only in Phase 2.
- Do not accept a client-supplied workspace_id as the sole authorization proof.
- Before modifying/deleting data, re-check that the entity belongs to the authenticated workspace.

5.5 Row Level Security Policies

Even where the application layer already performs authorization, Neon PostgreSQL must enable RLS as defense in depth. During implementation, ensure the database connection role does not bypass RLS and securely pass the current user identity into the transaction/session, for example through a controlled `app.clerk_user_id` setting; do not accept arbitrary, unverified client values.

- `users` : Users may only `SELECT/UPDATE` their own profile.
- `workspaces` : Owners may only `SELECT/INSERT/UPDATE/DELETE` their own workspace.
- `applications` : Access is limited to applications owned by the user's workspace.
- `requirements` : Access is limited to rows belonging to applications in the user's workspace.
- `tasks` : Access is limited to rows belonging to applications in the user's workspace.
- `documents` : Access is limited to documents in the user's workspace; users may delete only documents they uploaded or documents the workspace owner permits them to delete.
- `application_documents` : Operations are permitted only when both the application and document belong to the same workspace.
- `subscriptions` : The workspace owner may only read their own subscription; writes may be performed only by the server-side billing service role.
- `consents` : Users may only read their own consent history; inserts are written by the application server, and withdrawals must retain history.
- `stripe_events` : Access is permitted only to the server-side webhook role; ordinary authenticated roles may not read this table.

RLS policies must be verified through automated integration tests: User A must not be able to read, update, or delete any workspace data belonging to User B.

---

6. Third-Party API & Payment Boundaries

6.1 Required `.env.local` Keys

The following keys may exist only in the local environment / secret manager and must never be committed to git:

Plain Text

```
# Application
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

```
# Neon / Drizzle
DATABASE_URL=

# Clerk
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=
CLERK_SECRET_KEY=
CLERK_WEBHOOK_SECRET=

# Stripe
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
STRIPE_MONTHLY_PRICE_ID=
STRIPE_YEARLY_PRICE_ID=

# ImageKit
NEXT_PUBLIC_IMAGEKIT_URL_ENDPOINT=
NEXT_PUBLIC_IMAGEKIT_PUBLIC_KEY=
IMAGEKIT_PRIVATE_KEY=

# Gemini — Phase 2, server-side only
GEMINI_API_KEY=
```

- `NEXT_PUBLIC_*` may contain only public or frontend-required configuration; do not apply this prefix to private secrets.
- In production, use the hosting provider's secret manager; do not use plaintext files outside repository secrets.
- Provide `.env.example`, but keep all values blank or as placeholders.
- CI must perform secret scanning.

6.2 Clerk Boundary

- Use Clerk UI/components or the official server SDK.
- Protect `/app/*` and mutation APIs using Clerk middleware on the server side.
- Clerk webhooks, if used, are only for synchronizing user profiles; verify the webhook signature.
- Do not store Clerk passwords, MFA secrets, or session tokens.

6.3 ImageKit Boundary

- The browser must never receive the ImageKit private key.

- The client requests short-lived upload authentication parameters from `/api/upload/imagekit`.
- The server validates Clerk identity, workspace ownership, MIME type, file size, and document type.
- Store the ImageKit file ID rather than relying only on a mutable URL.
- When deleting a document, have the server call the ImageKit delete API, then delete the local row; if external deletion fails, retain a retryable state and avoid losing metadata.
- The MVP restricts file types and sizes. The recommended initial policy allows only PDF, DOCX, PNG, and JPEG, with a maximum of 10 MB; implementation may make this configurable through environment settings.

6.4 Gemini API Boundary (Phase 2)

- Call Gemini only from a server-side service.
- By default, do not send complete personal documents to Gemini unless the user gives explicit, revocable AI-processing consent.
- The UI clearly indicates that input data will be sent to a third-party AI provider.
- AI output must be labeled as suggestions and must not be submitted automatically.
- Record the model, request purpose, consent state, and error; avoid recording the complete contents of sensitive documents.
- Set a timeout, rate limit, maximum token/input size, and error fallback.
- Add usage-credit analytics only in Phase 2; do not make unlimited hidden calls to Gemini.

6.5 Stripe Checkout Flow

1. The user selects monthly or yearly on `/pricing` or `/app/billing`.
2. The client calls the authenticated server route `/api/stripe/checkout`.
3. The server obtains the Clerk identity and workspace and confirms the workspace owner.
4. The server creates or reuses a Stripe Customer.
5. The server creates a Checkout Session using the server-side configured Price ID.
6. The session is configured with subscription mode, a first-month free trial, success URL, cancel URL, and customer metadata (`workspace_id` , `user_id`).
7. The server returns the Checkout URL; the client redirects to Stripe.

8. The user completes or cancels payment.
9. Stripe sends an event to `/api/stripe/webhook`.
10. The webhook verifies the `STRIPE_WEBHOOK_SECRET` signature, writes to `stripe_events`, and uses `stripe_event_id` for idempotency.
11. Update `subscriptions` according to the event.
12. `/checkout/success` displays "Payment submitted, synchronizing"; display active only after the server subscription state has been updated.

6.6 Required Stripe Webhook Events

At a minimum, handle:

- `checkout.session.completed`
- `customer.subscription.created`
- `customer.subscription.updated`
- `customer.subscription.deleted`
- `invoice.paid`
- `invoice.payment_failed`

The webhook handler must:

- Verify the Stripe signature using the raw request body.
- Support duplicate events and unordered event delivery.
- Check `stripe_events` first to prevent duplicate processing.
- Update subscription state in a transaction.
- Return 2xx for unknown event types and log safe event metadata, to avoid infinite Stripe retries.
- Do not write the complete payment payload to application logs.

6.7 Billing Access Rules

- `trialing`, `active`: Full MVP access.
- `past_due`: Display a billing warning; a short read/write grace period may be retained according to a business decision.
- `canceled` or `incomplete_expired`: Retain data, restrict creation/editing, and provide a resubscribe CTA.

- When webhook status and frontend queries disagree, the webhook-synchronized server state prevails.

---

7. Compliance & Security

7.1 WCAG 2.2 Level AA

- The contrast ratio between all normal text and its background is at least **4.5:1**.
- Large text is at least **3:1**, although the product should still aim for 4.5:1 by default.
- Every interactive element must have a visible keyboard focus ring: `outline: 2px solid`, and it must not be clipped by the layout.
- Do not rely only on color to communicate status, error, required fields, or deadline urgency; pair it with an icon, text, or pattern.
- All form controls must have programmatic labels, error messages, and required indications.
- All functionality is keyboard reachable; there is no keyboard trap.
- Modals, dropdowns, tabs, toasts, and popovers must have appropriate focus management, ARIA semantics, and Escape behavior.
- Images provide meaningful alt text; decorative images use empty alt text.
- If drag-and-drop exists, provide a non-drag alternative.
- Responsive layouts must not lose content or functionality when zoomed to 200%.
- Text, buttons, and statuses remain readable and have sufficient target sizes on mobile.
- Use `prefers-reduced-motion` to reduce non-essential animation.
- Error messages must state how to fix the issue; they must not only display “Something went wrong”.
- Validate using axe, Lighthouse, keyboard-only testing, and a screen-reader smoke test.

7.2 GDPR / Hong Kong PDPO Privacy Requirements

Privacy Notices and PICS

- Provide a PICS / Personal Information Collection Statement at appropriate points, including registration, initial onboarding, document upload, and AI processing (Phase 2).

- The PICS must explain in clear language:
  - Which personal data is collected.
  - The purpose of collection and how the data is used.
  - Which data is mandatory or voluntary to provide.
  - Whether data will be provided to data processors such as Stripe, ImageKit, Clerk, and Gemini.
  - The retention period or the criteria used to determine it.
  - Channels for access, correction, deletion, consent withdrawal, and complaints.
  - Information relevant to cross-border transfers or third-party processing.
- Where GDPR applies, include controller/processor roles, lawful basis, data-subject rights, and DPA processes in the Privacy Policy.
- Do not obtain optional marketing consent through a pre-selected checkbox.

Consent Records

- Record required consent separately from optional marketing consent.
- For each consent, store `policy_version`, `granted`, `consented_at`, and, where applicable, `withdrawn_at`.
- When consent wording changes, create a new policy version; do not overwrite prior records.
- Consent withdrawal does not affect lawful processing before withdrawal, but all future optional processing must stop.
- Obtain explicit AI-processing consent before transmitting documents to Gemini (Phase 2).

Data Minimization and Retention

- Collect only the data needed to create and manage an application plan.
- Do not require users to provide passport numbers, bank information, third-party account passwords, or other unnecessary highly sensitive data.
- Retain documents and application data only for the shortest necessary period; provide an account-deletion/data-export process (the MVP must at minimum offer an actionable deletion-request path).
- On account deletion, delete local rows and ImageKit objects through a job/queue, while retaining legally required billing/consent audit records with only the necessary fields.

- Do not write complete CV, Personal Statement, or recommendation-letter contents to logs, analytics, or error trackers.

User Rights

Provide a process for initiating a request or contacting support to handle:

- Access / data export.
- Rectification.
- Erasure.
- Restriction / objection.
- Consent withdrawal.
- Marketing opt-out.
- Data breach notification and internal incident escalation.

7.3 Security Controls

- **Zero hardcoded secrets:** Source code, client bundles, SQL migrations, logs, and README files must not contain real secrets.
- Use the Stripe webhook signature, Clerk webhook signature, and official verification mechanisms from other providers.
- Enforce HTTPS (except for localhost development).
- Configure secure, httpOnly, sameSite cookies (managed by the official authentication SDK).
- Server-side input validation, output escaping, and CSRF protection (where applicable for cookie-based mutations).
- Apply rate limits to authentication, upload, AI, checkout, and mutation APIs.
- Prevent IDOR: for every entity retrieved by URL `id` , re-confirm workspace ownership.
- Do not trust client-side `subscription status` , `user_id` , `workspace_id` , or pricing.
- Error responses must not expose SQL, provider secrets, stack traces, or internal identifiers.
- Configure Neon, Stripe, ImageKit, and Gemini keys according to least privilege.
- Regularly run dependency audits, secret scans, SAST, and migration reviews.
- Establish a security-incident runbook covering token rotation, webhook replay, data-breach triage, and notification procedures.

7.4 Observability

- Log only necessary structured fields: request ID, route, status, duration, safe entity ID, and error code.
- Stripe webhook logs record event ID/type, not complete card/payment payloads.
- Monitor 5xx rate, webhook failures, checkout failures, upload failures, database latency, and unauthorized access attempts.
- Redact or hash any PII-sensitive log fields.

---

8. Acceptance Criteria

Core Product

- [ ] Users can register, sign in, and sign out with Clerk.
- [ ] On first login, users can select and save `career` or `masters` mode.
- [ ] Users can view and modify only their own workspace data.
- [ ] Users can create, edit, and delete applications.
- [ ] The server rejects writes when the application type does not match workspace mode.
- [ ] Users can create requirements, tasks, and documents.
- [ ] The dashboard correctly calculates applications, tasks, and deadlines.
- [ ] The compare page can compare at least two applications.
- [ ] Documents upload through ImageKit, with no private-key exposure.

Billing

- [ ] The pricing page displays monthly/yearly options and the first-month-free terms.
- [ ] The server creates a Checkout Session using the configured Stripe Price ID.
- [ ] Webhook signatures must pass verification.
- [ ] Resending the same Stripe event does not create duplicate data or incorrectly overwrite data.
- [ ] Subscription status is updated by webhooks and drives the access gate.
- [ ] Data is not immediately deleted after the user cancels or a payment fails.

Accessibility and Privacy

- [ ] Text contrast reaches 4.5:1.
- [ ] The focus ring is `2px solid` and visible during keyboard interaction.
- [ ] The core journey can be completed entirely with the keyboard.
- [ ] PICS, Privacy Policy, and Terms are visible in the appropriate flows.
- [ ] Consent timestamps, policy version, and withdrawal state are traceable.
- [ ] No hardcoded secrets exist.
- [ ] User A's RLS/integration test cannot read User B's data.

---

9. Recommended Implementation Order

1. Set up Next.js App Router, TypeScript, Tailwind, and strict linting.
2. Configure Clerk, then create the protected layout and user synchronization.
3. Create the Neon/Drizzle schema, migrations, indexes, and RLS.
4. Implement onboarding, workspace mode, and consent recording.
5. Implement application CRUD, requirements, and tasks.
6. Implement the dashboard, deadline sorting, progress calculation, and comparison UI.
7. Integrate ImageKit upload/authentication and document linking.
8. Implement pricing, Stripe Checkout, webhook idempotency, and the access gate.
9. Complete privacy pages, the PICS flow, and the account/data deletion-request path.
10. Conduct accessibility, security, RLS-isolation, billing-webhook, and responsive QA.
11. Verify all environment variables, deployment secrets, migrations, and rollback procedure.
12. Retain Gemini AI and Phase 2 capabilities behind a feature flag or in a separate backlog; do not block the MVP launch.

---

10. Launch Definition of Done

GradMap may enter production launch only after all of the following conditions are met:

- [ ] Phase 1 scope is complete, and Phase 2 functionality has not been mixed into the core release.
- [ ] Production database migrations have been rehearsed, with a rollback or recovery plan.

- [ ] Clerk, Neon, Stripe, ImageKit, and deployment secrets are configured in a secret manager.
- [ ] Stripe test-mode end-to-end tests pass, including trial, successful payment, cancellation, payment failure, and webhook replay.
- [ ] RLS-isolation tests pass.
- [ ] The accessibility audit has no unresolved critical/serious issues.
- [ ] GDPR/PDPO privacy notices, PICS, consent records, and retention decisions have been reviewed by the appropriate legal/privacy owner.
- [ ] The CI check for no hardcoded secrets passes.
- [ ] Error tracking, safe logs, rate limiting, and an incident contact are configured.
- [ ] Users can complete onboarding and create their first application plan within 10 minutes.